

一维数组

复习

综合案例1

循环的应用

分支结构的应用等

目录

- 1. 前言
- 2. 一维数组的定义和初始化 重点
- 3. 一维数组数据元素的访问 重点
- 4. 一维数组的应用案例-冒泡排序 熟悉
- 5. 断点调试 重点

情景引入

保存一个的年龄，我们可以用 `int age = 20 ;`

保存一个班级的学生年龄呢？需要大概30个整型变量

保存100 0000个年龄呢？



太多了，咋整？

前言

在程序开发的过程中，很多场景，我们需要保存很多的数据，或者说我们需要保存一组数据，使用普通的数据类型是不能满足我们需求的，C++ 为开发者提供了数组用来保存一组数据。

在 C++ 中，数组保存的一组数据的类型必须是完全一致的，C++ 不支持数组保存不同的数据类型。

一维数组的定义和初始化

定义数组

语法

数据类型 数组名[数组长度];

int arr_num[5];

数据类型 数组名[数组长度] = {数据1 , 数据2 , ...};

int arr_num2[5] = {1,2,3,4,5};

数组名

- 1.只能由字母、数字和下划线组合，但数字不能开头
- 2.系统关键字不能作为数组名 `int cout cin`等
- 3.同一作用域下不能重名
- 4.见其名知其意

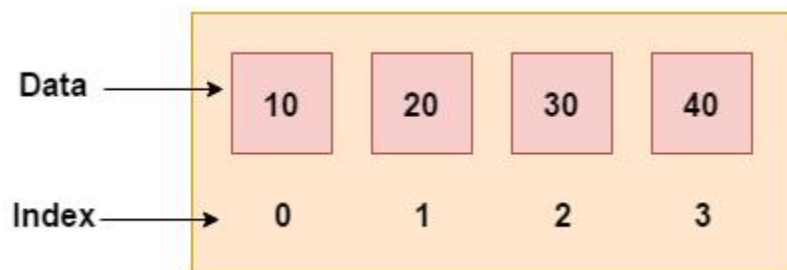
数组名一般以ar或者arr开头
比如：`arr_num ar_score`

数组下标

数组是一组具有连续内存位置的类似类型的元素。

在C++中`std::array`是一个封装固定大小数组的容器。

在C++中，数组索引从0开始。我们可以在C++数组中只存储固定的元素集合。



赋值 取值

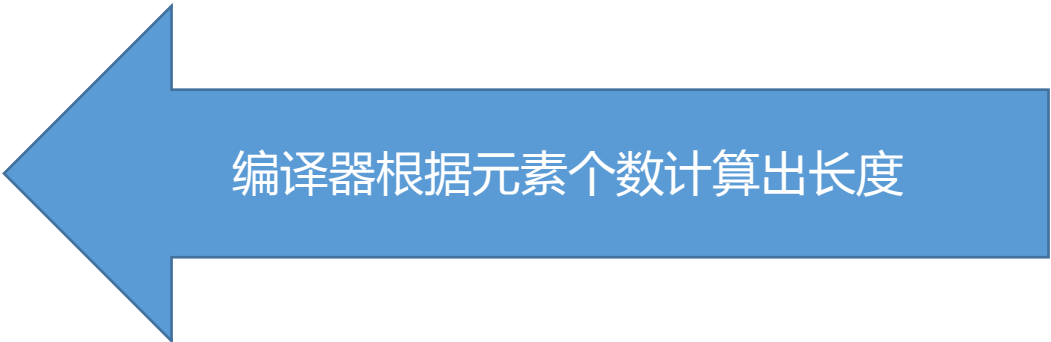
数组初始化

```
int ar1[5] = {100,200,300,400,500};
```

```
int ar2[5] = {10,20};
```

```
int ar3[5] = {};
```

```
int ar4[] = {1,2,3,4,5};
```



编译器根据元素个数计算出长度

一维数组数据元素的访问

访问数组的元素，使用中括号，即 [] 加上数组索引即可。

但是需要注意的是，C++ 数组的索引是从 0 开始的。

因此，如果我们需要访问数组的第一个元素，其索引是 0，要访问数组的最后一个元素，其索引是数组的长度减一。

语法

数组名[索引];

一维数组数据元素的访问

```
int arr[10];  
arr[0] = 111;  
cout << arr[0] << endl;
```

```
int arr2[5] = {1,2,3,4,5};  
cout << arr2[0] << arr2[1] << arr2[2] << arr2[3] << arr2[4] ;
```

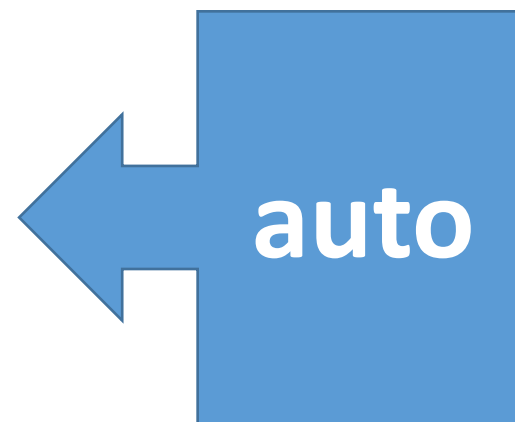
有没有更高效的访问多个元素的方法呢？

循环访问

```
int arr[5] = {1,2,3,4,5};  
for(int i=0; i<5; i++){  
    cout << arr[i] << " ";  
}
```

使用foreach循环遍历

```
int arr[5] = {1,2,3,4,5};  
for(int i: arr){  
    cout << i << " ";  
}
```



循环赋值

```
int arr[5] = {1,2,3,4,5};  
for(int i=0; i<5; i++){  
    cin >> arr[i] ;  
}
```

数组长度

1.整型常量

2.整型常量

(1) #define

```
#define N 100
```

(2) const

```
const int N = 100;   int const N = 100;
```

练习1

现有五只小猪，体重分别为764，942，613，476，940。定义一个一维数组，存放五只小猪的体重，找出其中最重的小猪，并打印输出。

练习1 代码

```
const int N = 5;
int arr[N];
for (int i = 0; i < N; i++)
{
    cin >> arr[i];
}
int max_weight = arr[0];
for (int i = 0; i < N; i++)
{
    if (arr[i] > max_weight) {
        max_weight = arr[i];
    }
}
cout << max_weight << endl;
```

练习2

输入 n ($n \leq 100$)个数保存到数组中，求 n 个数的和以及平均值。

练习2 代码

```
int arr[100], n;
cin >> n;
for (int i = 0; i < n; i++)
{
    cin >> arr[i];
}
int sum = 0;
for (int i = 0; i < n; i++)
{
    sum += arr[i];
}

cout << sum << endl << sum * 1.0 / n << endl;
```

比较两个数组是否相等

不能使用 `==` 运算符与两个数组的名称来确定数组是否相等。

```
int arr1[5] = {1,2,3,4,5};  
int arr2[5] = {1,2,3,4,5};  
if(arr1 == arr2){  
    cout << “相等” << endl;  
}  
else{  
    cout << “不相等” << endl;  
}
```



```
int arr1[5] = { 11, 13, 50, 72, 91 };
int arr2[5] = { 11, 13, 50, 72, 91 };
int arr1Count = sizeof(arr1) / sizeof(arr1[0]);
int arr2Count = sizeof(arr2) / sizeof(arr2[0]);
if (arr1Count != arr2Count)
{
    cout << "arr1 != arr2" << endl;
    return 0;
}
for (int i = 0; i < arr1Count; i++)
{
    if (arr1[i] != arr2[i])
    {
        cout << "arr1 != arr2" << endl;
        return 0;
    }
}
cout << "arr1 == arr2" << endl;
```



下面数组如何比较？

```
int arr1[5] = { 50, 13, 11, 72, 91 };  
int arr2[5] = { 91, 11, 72, 50, 13 };
```



good idea ?

一维数组的应用案例-冒泡排序

在实际生活或者工作中，很多数据都需要排序；

比如考试成绩排序；

比如价格排序；

比如重量排序等。

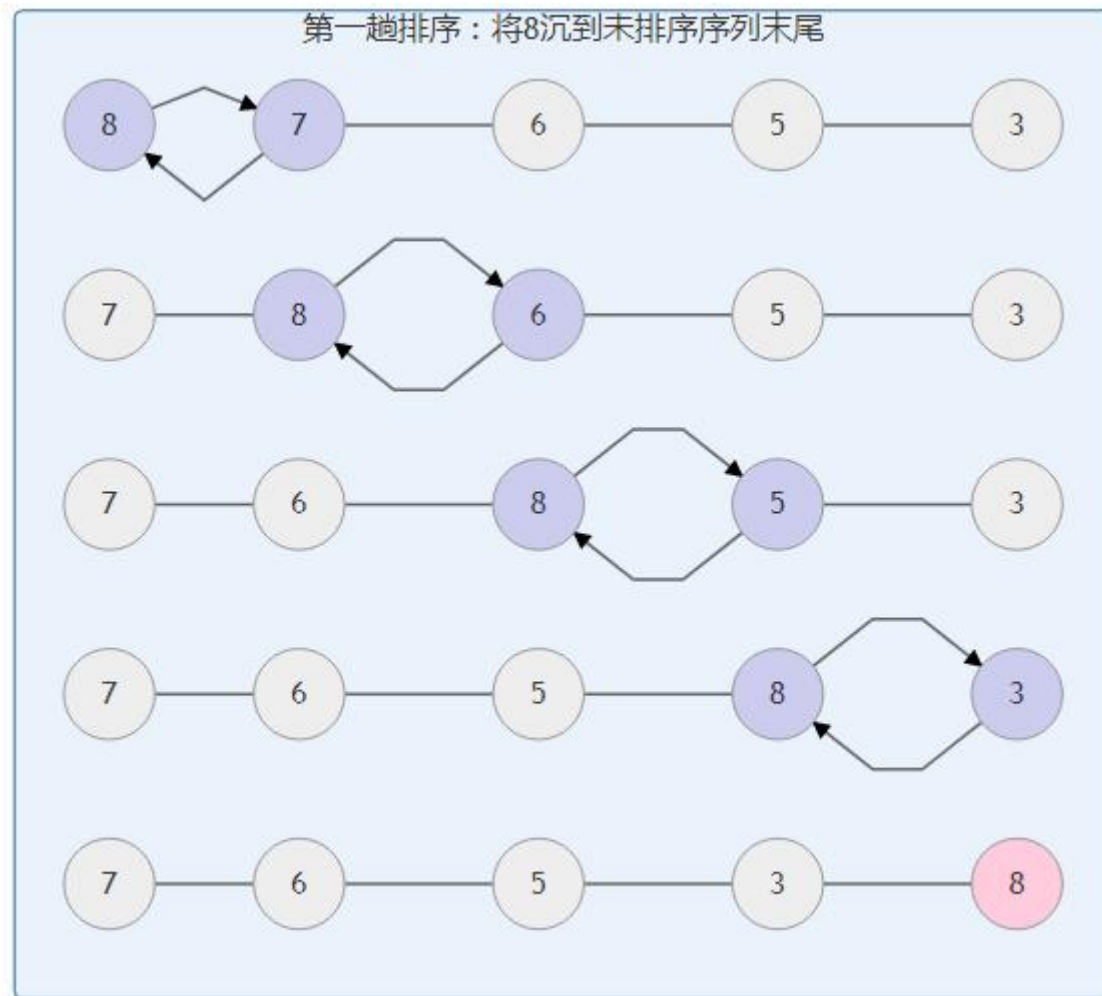
理解并实现冒泡排序算法

- 1.比较相邻的元素。如果第一个比第二个大，就交换他们两个。
- 2.对每一对相邻元素做同样的工作，执行完毕后，找到第一个最大值。
- 3.重复以上的步骤，每次比较次数-1，直到不需要比较。

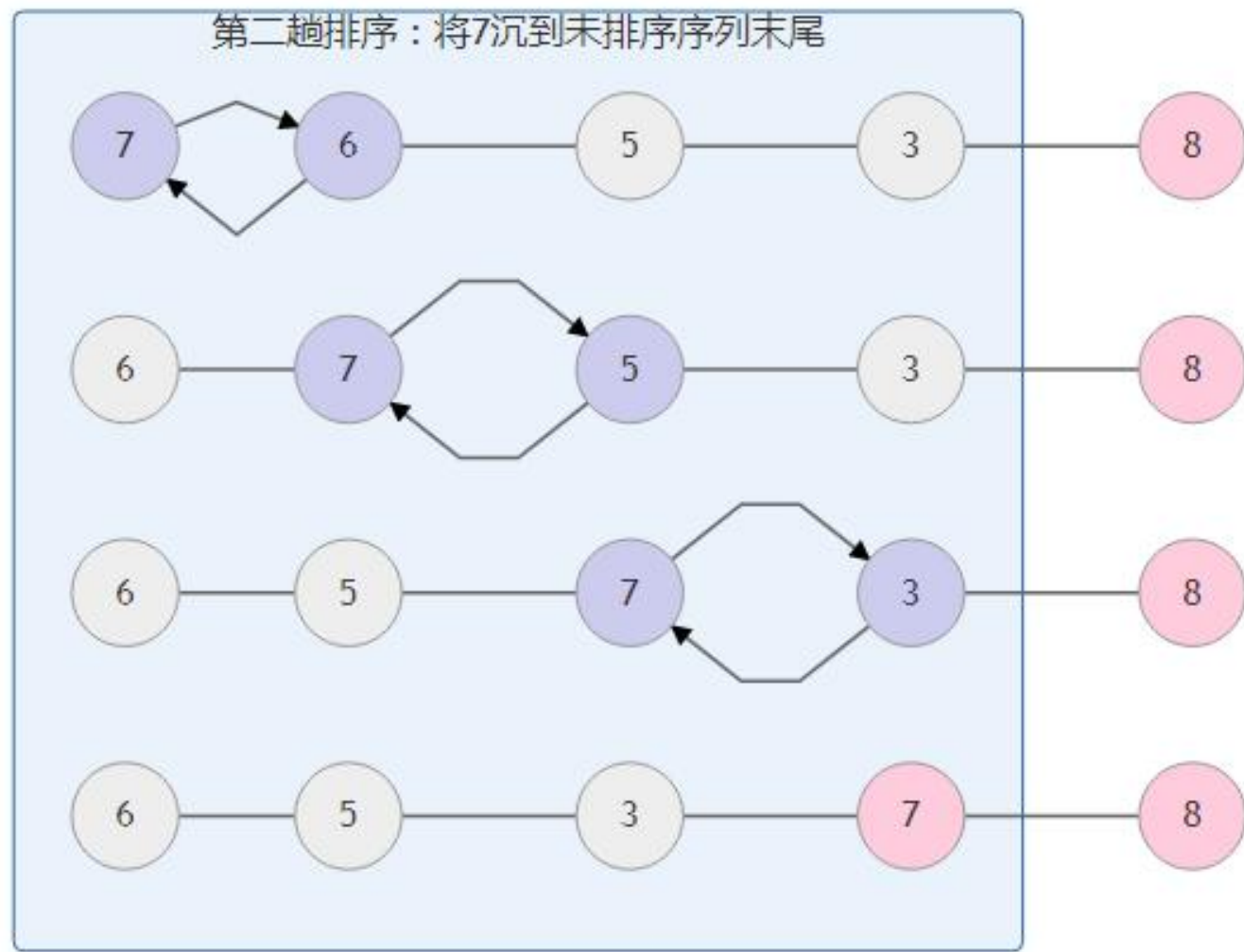
冒泡排序图解



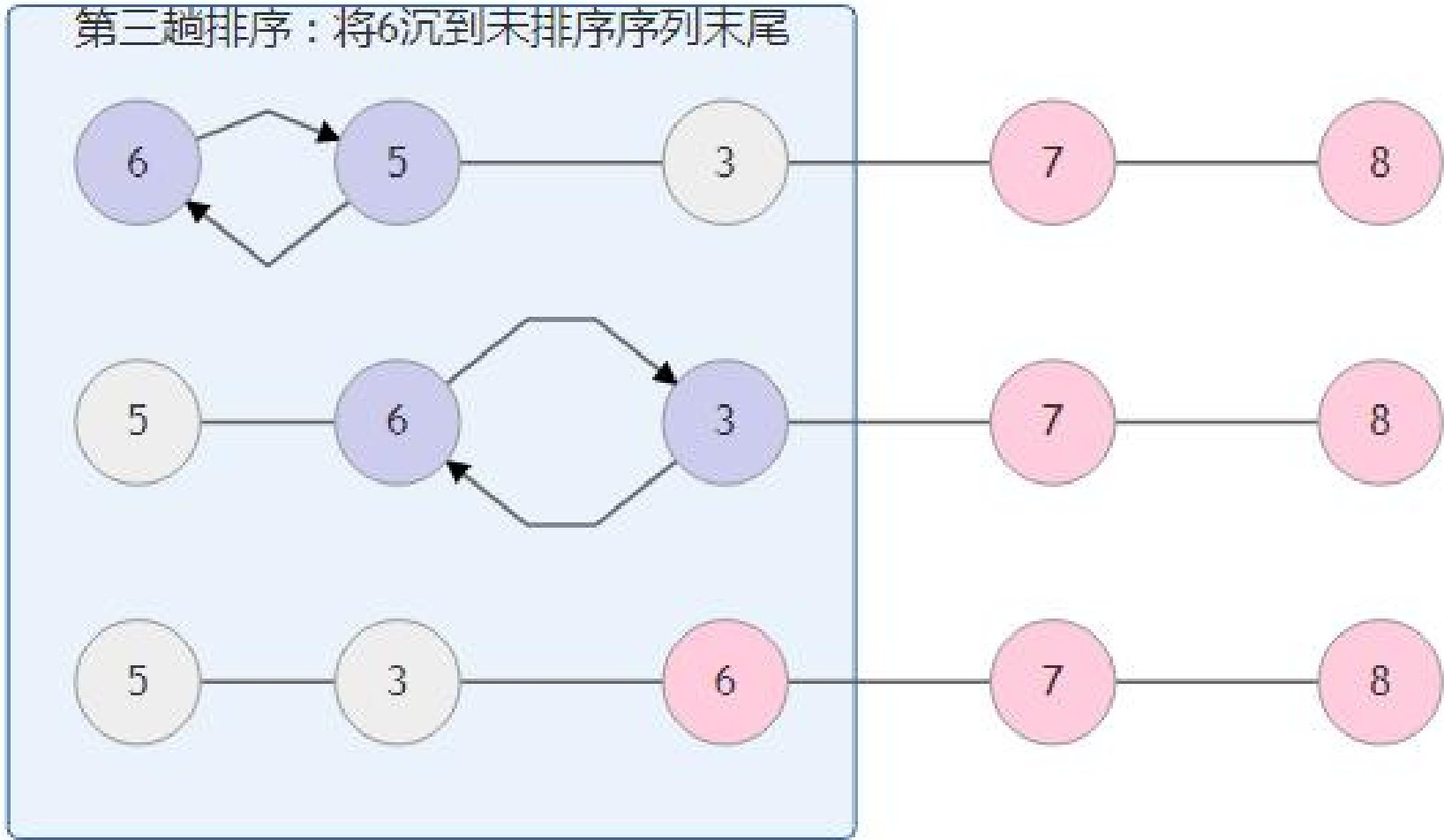
第一趟



第二趟

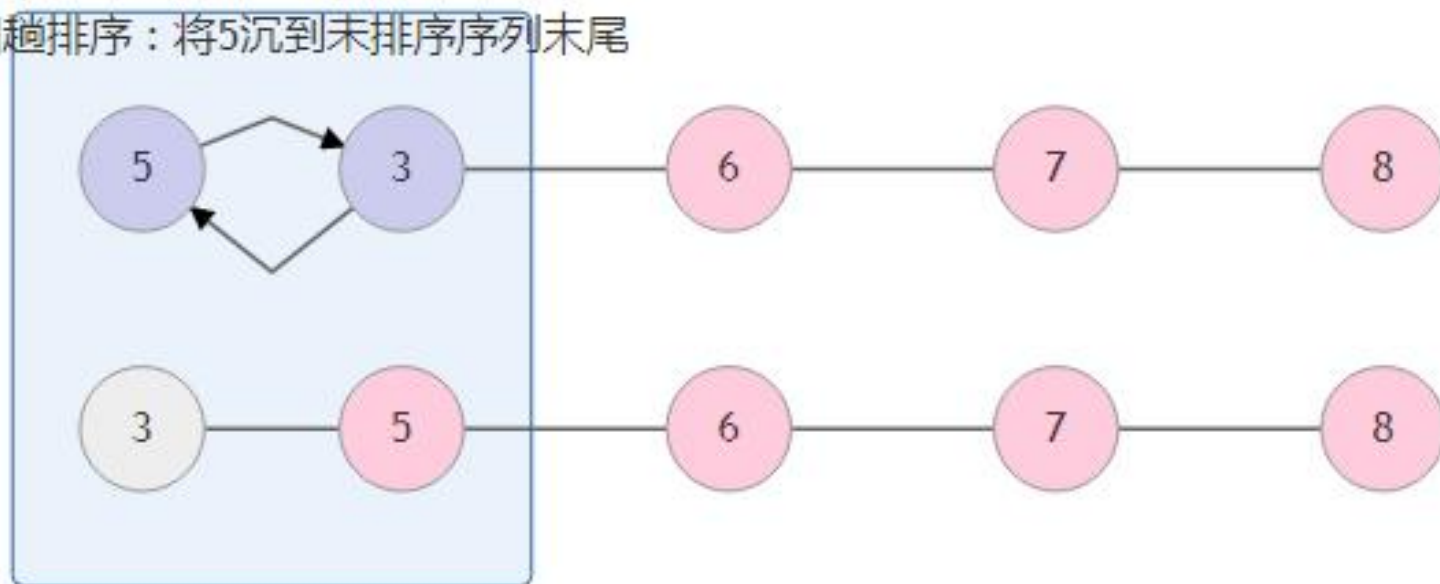


第三趟

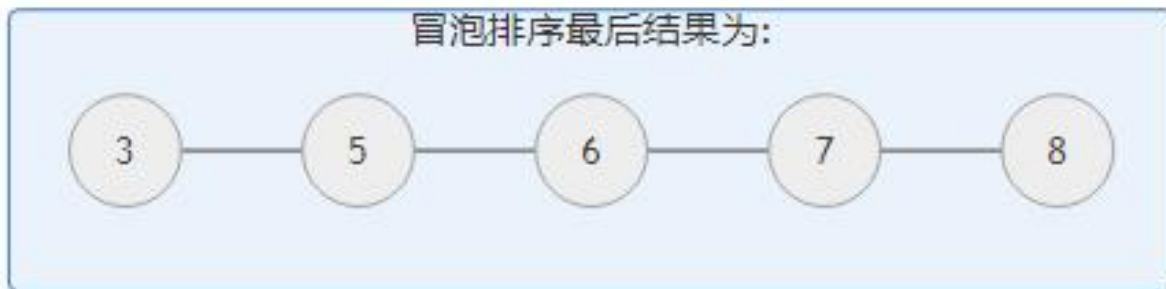


第四趟

第四趟排序：将5沉到未排序序列末尾



冒泡排序最后结果为：



交换数据

```
int a = 1 ;
```

```
int b = 2 ;
```

输出 : a = 2 b = 1

你能想到几种方法？

交换数据

```
int a = 1 ;
```

```
int b = 2 ;
```

输出 : a = 2 b = 1

- 
- 1.临时变量
 - 2.加减法
 - 3.使用系统函数

冒泡排序完整代码

```
int arr[N] = { 8,7,6,5,3 };  
//趟数  
for (int i = 0; i < N-1; i++) {  
    //比较次数  
    for (int j = 0; j < N - i - 1; j++) {  
        if (arr[j] > arr[j + 1]) {  
            swap(arr[j], arr[j + 1]);  
        }  
    }  
}  
for (int i = 0; i < N; i++)  
{  
    cout << arr[i] << " ";  
}
```

怎么优化代码？

代码优化

```
int arr[N] = { 8,7,6,5,3 };
bool flag = true;
//趟数
for (int i = 0; i < N - 1; i++) {
    //比较次数
    flag = true;
    for (int j = 0; j < N - i - 1; j++) {
        if (arr[j] > arr[j + 1]) {
            swap(arr[j], arr[j + 1]);
            flag = false;
        }
    }
    if (flag) break;
}
for (int i = 0; i < N; i++)
{
    cout << arr[i] << " ";
}
```



增加标记变量

断点调试

为什么需要断点调试？

在开发中，新手程序员在查找错误时,这时老程序员就会温馨提示，可以用断点调试，一步一步的看源码执行的过程，从而发现错误所在。

1.断点调试是指在程序的某一行设置一个断点，调试时，程序运行到这一行就会停住，然后你可以一步一步往下调试，调试过程中可以看各个变量当前的值，出错的话，调试到出错的代码行即显示错误,停下。进行分析从而找到这个Bug

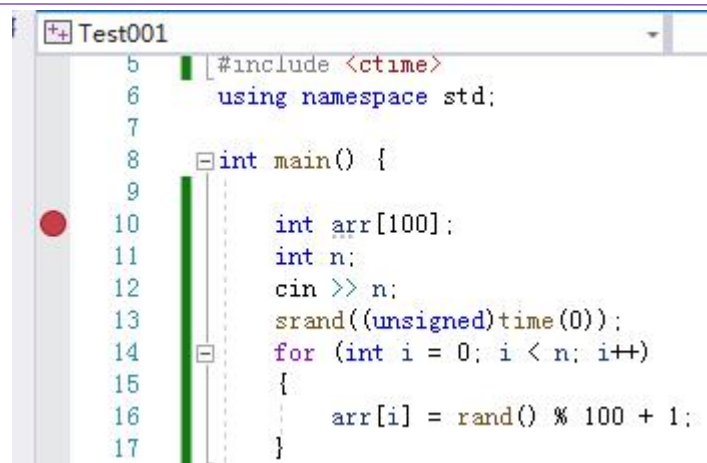
2.断点调试是程序员必须掌握的技能。

3.断点调试也能帮助我们查看底层源代码的执行过程，提高程序员的编程水平。

VS断点调试 步骤

1. 打断点

鼠标在行序号前点击即可



2. 点击调试-开始调试 或直接按F5快捷键

3. 进入调试状态，进行调试



VS调试

VS Debug Interface showing the execution of `Test001.exe` (Process: [9960], Thread: [9964] Main Thread, Stack Frame: main).

Test001.cpp (Global Scope) - `main()`

```
#include <ctime>
using namespace std;

int main() {
    int arr[100];
    int n;
    cin >> n;
    srand((unsigned)time(0));
    for (int i = 0; i < n; i++)
    {
        arr[i] = rand() % 100 + 1;
    }
    已用时间 <= 6ms
}
```

诊断工具 (Diagnostic Tools)

诊断会话: 6 秒 (已选择 6.706 秒)

摘要 事件 内存使用率 CPU 使用率

筛选器 筛选事件

事件	时间	持续时...	线程
断点: Test001.cpp...	0.77 秒	778毫秒	[9964]
步骤: Test001.cpp...	6.67 秒	5,897毫秒	[9964]
步骤: Test001.cpp...	6.69 秒	22毫秒	[9964]
步骤: Test001.cpp...	6.69 秒	5毫秒	[9964]
步骤: Test001.cpp...	6.70 秒	6毫秒	[9964]

自动窗口 (Auto Window)

搜索 (Ctrl+E) 搜索深度: 3

名称	值	类型
arr	0x00000000016f4a0 {63, -858993460, -858993460, -85...	int[100]
arr[i]	63	int
i	0	int
n	5	int
rand	0x000000013ff81979 {Test001.exe!rand}	void *

调用堆栈 (Call Stack)

名称	语言
Test001.exe!main() 行 17 [外部代码]	C++

调用堆栈 断点 异常设置 命令窗口 即时窗口 输出

断点调试练习

用断点调试，找出下面代码的问题。正常循环输出数组中的元素。

```
int arr[6] = { 1,2,3,4,5,6 };  
int index = 0;  
while (index < 6) {  
    cout << arr[index] << " ";  
}
```

课堂练习

随机 n ($n \leq 100$)个1-100数字，保存到数组中，然后用冒泡排序进行排序，输出降序序列。

课堂练习 代码

```
int arr[10];
int n;
cin >> n;
srand((unsigned)time(0));
for (int i = 0; i < n; i++)
{
    arr[i] = rand() % 100 + 1;
}

for (int i = 0; i < n - 1; i++)
{
    for (int j = 0; j < n - i - 1; j++) {
        if (arr[j] > arr[j + 1]) {
            swap(arr[j], arr[j + 1]);
        }
    }
}

for (int i = 0; i < n; i++)
{
    cout << arr[i] << " ";
}
```

知识点总结

1. 前言
2. 一维数组的定义和初始化 重点
3. 一维数组数据元素的访问 重点
4. 一维数组的应用案例-冒泡排序 熟悉
5. 断点调试 重点

作业

1. 利用冒泡排序，完成下面数组排序

```
int arr[10] = {99,11,2,3,4,6,9,7,100,1}
```

下节课预告

一维数组的常用操作